

WBE-Praktikum 1 (Ergänzung)

Entwicklungsumgebung

Einführung

In dieser Beschreibung geht es vor allem darum, die benötigte Infrastruktur einzurichten und ein paar Versuche damit zu machen. Es ist gut möglich, dass Sie einige der Werkzeuge bereits für andere Kurse, eigene Web-Projekte, oder im Rahmen des Vorkurses installiert und verwendet haben.

Bitte betrachten Sie die in den folgenden Abschnitten angegebenen Werkzeuge als Vorschläge und nicht als Vorgaben. Wenn Sie bereits einen Code-Editor im Einsatz haben, welcher auch die Web-Sprachen HTML, CSS und JavaScript beherrscht (das wird normalerweise der Fall sein), können Sie diesen selbstverständlich verwenden. Das gleiche gilt für die Auswahl der eingesetzten Browser. Viele Hinweise in den Folien und Praktikumsaufgaben beziehen sich allerdings auf den im folgenden vorgeschlagenen Visual Studio Code (und speziell auf Erweiterungen für dieses Programm) und den Firefox mit seinen Entwicklerwerkzeugen.

Ergänzend gibt es einen Foliensatz mit einer Zusammenfassung der wichtigsten Tools. Am besten schauen Sie Folien und die untenstehenden Aufgaben durch und entscheiden je nach Vorkenntnissen, welche Teile neu für Sie sind. Dann können Sie sich auf diese Aufgaben beschränken.

Browser mit Entwicklertools

Als Web-Entwickler sollte man verschiedene Browser zum Testen zur Verfügung haben. Es bringt aber wenig, Aufwand zum Testen mit verschiedenen Browsern zu treiben, wenn diese auf der gleichen Rendering Engine basieren: Chrome, Opera, Vivaldi und der neue Edge verwenden die Rendering Engine Blink von Google, grosse Unterschiede in der Darstellung sind also nicht zu erwarten. Hier ein paar Tipps:

- **Chrome** (Google) ist heute der am weitesten verbreitete Browser. Wer Bedenken hat, den Browser einer Firma zu verwenden, welche den grössten Teil ihres Umsatzes mit Werbung macht (und damit zwangsläufig auch mit dem Sammeln von Daten über Sie), kann **Chromium** verwenden, das Open-Source-Projekt, auf dem der Chrome basiert. Chrome und Chromium haben sehr gute Entwicklertools und sind sehr schnell darin, neue Web-Standards zu unterstützen (welche nicht selten von Google-Entwicklern selbst vorangetrieben werden).

- **Firefox** (Mozilla) ist unter Windows praktisch die einzige verbliebene Alternative zu Browsern, die auf Chromium aufbauen. Gute Unterstützung aktueller Web-Standards. Ebenfalls mit sehr guten Entwicklertools und einer grossen Sammlung von verfügbaren Erweiterungen. Empfehlung: Firefox mindestens als Ergänzung zu anderen Browsern installieren.
- **Edge** (Microsoft) verwendet mittlerweile auch die Rendering Engine des Chromium-Projekts. Der alte, nicht auf Chromium basierende Edge spielt heute keine Rolle mehr.
- **Safari** (Apple) ist unter iOS und MacOS vorinstalliert und dementsprechend auf diesen Plattformen ein wichtiger Browser. Unter iOS ist die Webkit-Engine des Safari sowieso Vorschrift, das heisst auch Chrome oder Firefox verwenden unter iOS Apple's Webkit-Engine.

Neben der Hauptversion der Browser gibt es auch noch experimentelle Vorabversionen, die neue Features implementieren, aber noch Fehler enthalten können. Um das eine oder andere neue Feature ausprobieren zu können, kann es nötig sein, eine solche Vorabversion zu installieren. Fast alles, was wir in WBE behandeln, wird aber in den Hauptversionen der aktuellen Browser funktionieren.¹



Erweiterungen: Nicht viele nötig, da die Entwicklertools bereits sehr gute Werkzeuge enthalten. Die Erweiterung *Web Developer*² ist schon etwas älter, wird aber noch gepflegt und enthält ein paar nützliche Hilfsmittel. Heutzutage kommt man im Web ausserdem kaum um einen Content-Blocker herum. Empfehlenswert hier *uBlock Origin*³.

¹ Bilder aus: Why are there Four Firefoxes? <https://www.youtube.com/watch?v=qQ1oQJJn1nQ>

² <https://chrispederick.com/work/web-developer/>

³ <https://github.com/gorhill/uBlock#ublock-origin>



Code-Editor mit Erweiterungen

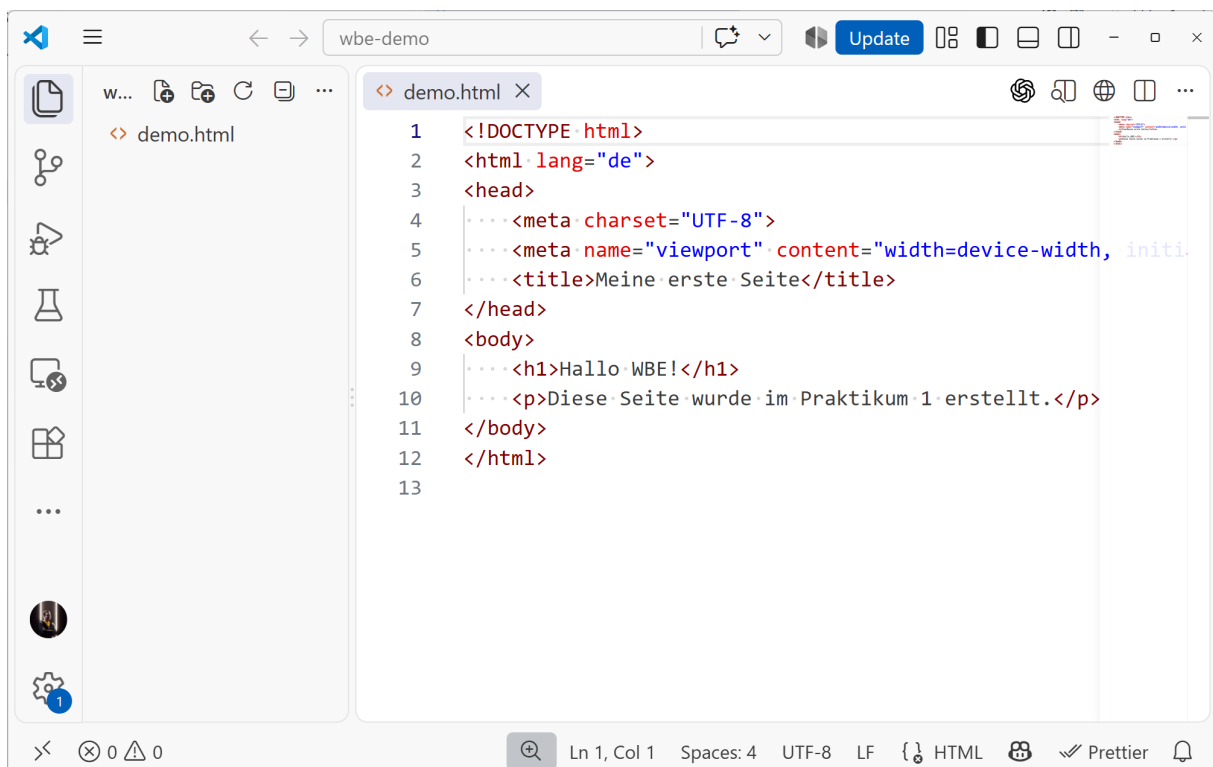
Selbstverständlich benötigen Sie zum Bearbeiten Ihrer JavaScript-Programme und von HTML- und CSS-Code einen geeigneten Code-Editor. Im Prinzip kann jeder Texteditor dafür eingesetzt werden. Mit Vorteil bietet der Editor aber zusätzliche Unterstützung zur Code-Bearbeitung:

- Syntax-Elemente geeignet einfärben. So behält man den Überblick und findet Fehler schneller.
- Unterstützung beim Bearbeiten – etwa Vorschlägen passender Attribute oder Schliessen von Tags beim Bearbeiten von HTML-Code, oder Vervollständigen von CSS-Eigenschaften.
- Plugin-System, um den Editor bei Bedarf erweitern zu können: Module zum Überprüfen oder Erweitern von Code, Syntax zusätzlicher Sprachen, Live-Vorschau der aktuellen Seite, Funktionen zur Verwaltung des Projekts in einem Git-Repository, ...

Sie können in WBE gerne Ihren Lieblings-Code-Editor verwenden. Sollten Sie noch keinen haben, empfehlen wir den **Visual Studio Code** (<https://code.visualstudio.com>, Microsoft). Der VSCode ist mittlerweile stark verbreitet und selbst mit Webtechnologien aufgebaut: Basis für VSCode (und für viele andere Applikationen heutzutage) ist das Electron-Projekt, und dieses baut auf dem Chromium-Browser auf.

Es gibt natürlich diverse Alternativen: Zed (moderner, schneller Editor), Sublime Text (kostenpflichtig), WebStorm (JavaScript-IDE von JetBrains) und noch ein paar mehr. Die Beispiele in WBE werden sich aber meistens auf VSCode beziehen.

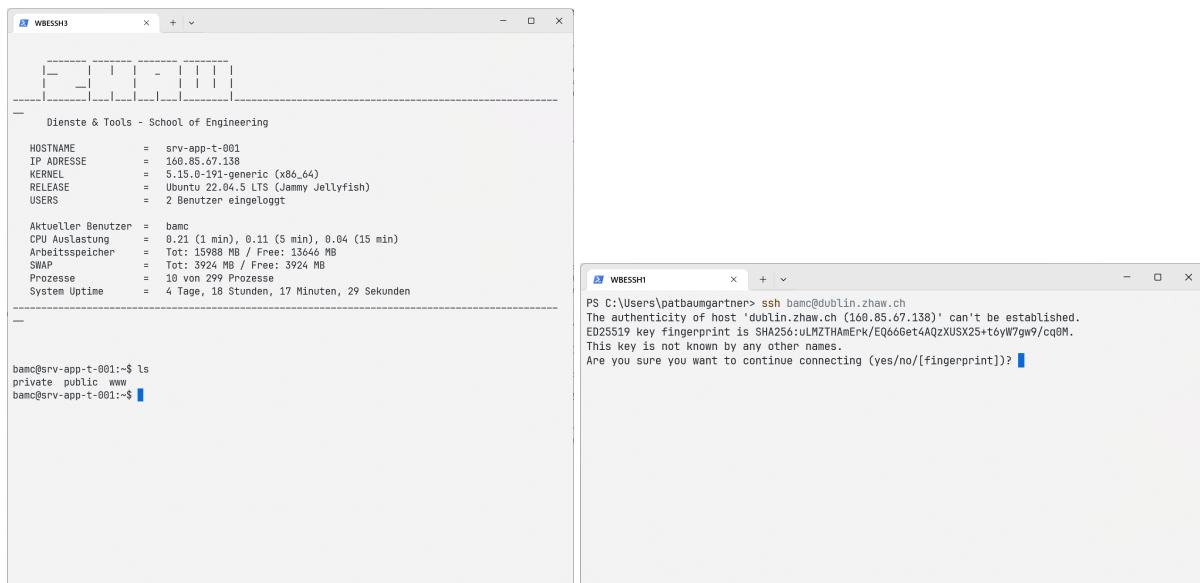
Erweiterungen für den VSCode: HTML, CSS und JavaScript bereits gut unterstützt. Ein paar Tipps zu Erweiterungen gibt's auf den Begleitfolien und nach Bedarf in den Lektionen.



SSH und SFTP

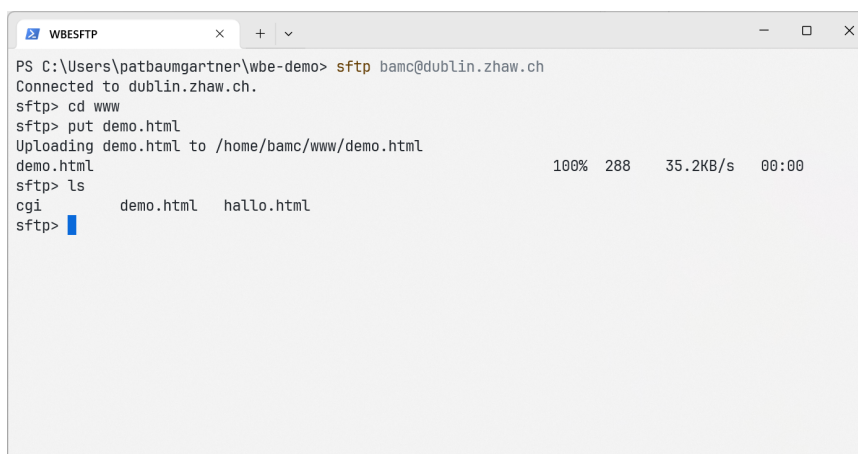
Bei der Entwicklung von Web-Applikationen ist es häufig nötig (oder mindestens hilfreich), sich auf einem Server einloggen und dort auf der Kommandozeile arbeiten zu können. Dazu kann das Kommandozeilen-Tool **ssh** verwendet werden. Es baut eine verschlüsselte Verbindung zu einem Server auf und öffnet dort eine *Shell*, welche Kommandozeilenbefehle entgegennimmt.

Unter MacOS und Linux steht **ssh** als Befehl im Terminal zur Verfügung. Unter Windows kommt es auf die Version an. Gegebenenfalls müssen Sie **ssh** dort erst aktivieren. Alternativ können Sie unter Windows auch Putty⁴ verwenden.



Bei der ersten Verbindungsaufnahme mit einem Server müssen Sie den Fingerabdruck des Servers bestätigen, um sicherzustellen, dass Sie mit dem korrekten Server verbunden sind.

Während SSH dazu dient, auf einem Server remote zu arbeiten, wird SFTP verwendet, um Dateien vom oder zum Server zu übertragen. Auch dafür gibt es ein Kommandozeilen-Tool: **sftp**.



⁴ <https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>

Alternativ können Programme mit grafischer Oberfläche verwendet werden, z.B. **FileZilla**⁵.

Wichtiger Hinweis zu FileZilla: Unter Windows enthält der Installer möglicherweise zusätzliche Angebote – «*This installer may include bundled offers*» – achten Sie darauf, nur den FileZilla-Client zu installieren.

⁵ Es gibt sicher Programme mit schönerem GUI als FileZilla, aber FileZilla hat sich als zuverlässig und robust erwiesen.



Node.js ist eine browserunabhängige JavaScript-Laufzeitumgebung basierend auf der V8-Engine, die auch im Chrome-Browser verwendet wird. Sowohl Node.js als auch die JavaScript-Umsetzungen der Browser basieren auf der von der Organisation ECMA definierten Spezifikation der Programmiersprache JavaScript. Die unterstützten JavaScript-APIs im Browser und in Node.js unterscheiden sich jedoch beträchtlich.

Für die JavaScript-Einführung im ersten Teil des Semesters werden wir Node.js verwenden. Für die Installation gibt es verschiedene Varianten. Empfohlen wird üblicherweise, Node.js mit einem Tool für die Node-Versionsverwaltung zu installieren, z.B. *nvm* oder *n* unter Linux oder MacOS, *nvm-windows* oder *fnm* unter Windows. Das hat den Vorteil, dass Sie Tests mit verschiedenen Node-Versionen machen können. Informationen dazu:

<https://docs.npmjs.com/downloading-and-installing-node-js-and-npm>

A screenshot of a Windows terminal window titled "WBENODE2". The terminal shows the following commands and output:

```
PS C:\Users\patbaumgartner> fnm install 22
Installing Node v22.23.2 (x64)
00:00:04 34.03 MiB/34.03 MiB (7.81 MiB/s, 0s)
PS C:\Users\patbaumgartner> fnm list
* v22.23.2 default
* system
PS C:\Users\patbaumgartner>
```

Falls das nicht klappt, können Sie auch das nötige Installationsprogramm direkt hier herunterladen:

<https://nodejs.org/en/download/>

Noch eine Variante: unter Linux und auf dem Mac können Sie Node.js auch mit einer Paketverwaltung installieren, auf dem Mac zum Beispiel mit *Homebrew* oder *Macports*:

```
$ # Homebrew
```

```
$ brew install node
```

```
$ # Macports
```

```
$ port install nodejs22
```

Mit der Installation von Node.js kommt normalerweise auch das Kommandozeilenprogramm *npm* für die Node.js-Paketverwaltung. Überprüfen Sie es durch Aufruf von *npm* auf der Kommandozeile. Es gibt

dann einfach einen Hilfetext aus. Falls *npm* nicht installiert ist, installieren Sie es separat.

Aufgaben

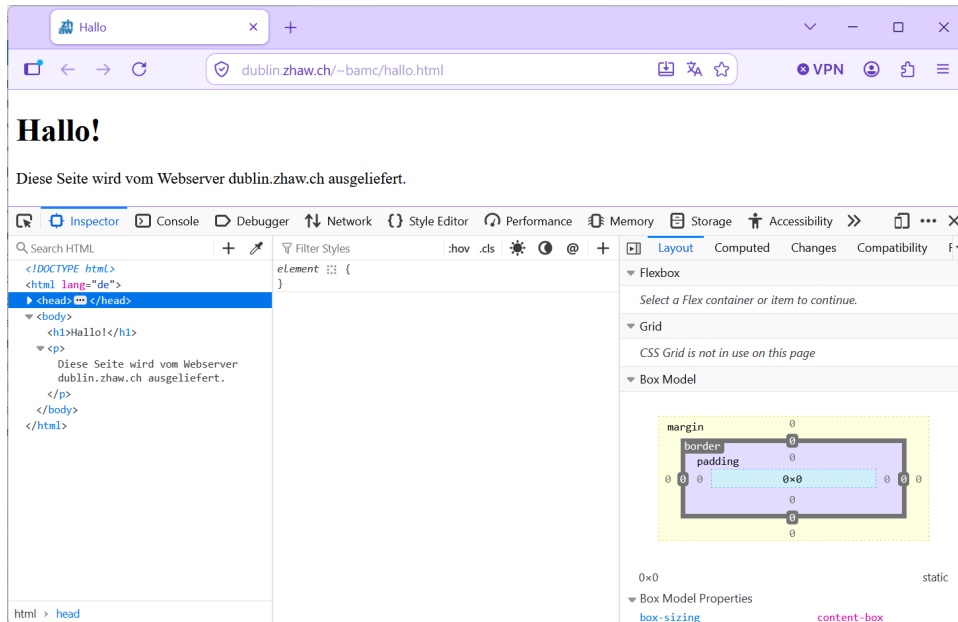
Es folgen einige Aufgaben zur Infrastruktur, welche Sie nach Bedarf und je nach Vorkenntnissen bearbeiten sollten, wenn Sie die nötigen Installationen abgeschlossen haben. In diesem Praktikum muss nichts abgegeben werden.

Aufgabe 1: Browser und Entwicklertools

Öffnen Sie eine einfache Website im Browser. Ein Beispiel finden Sie hier (nur aus dem ZHAW-Netz oder via VPN erreichbar):

<https://dublin.zhaw.ch/~bamc/hallo.html>

Öffnen Sie die Entwicklertools im Browser. Im Firefox sieht das folgendermassen aus:



Da die Seite kein JavaScript und keine Stylesheets enthält, sind die Möglichkeiten in den Entwicklertools für dieses Beispiel gering. Trotzdem eignet sich das Beispiel, um sich einen ersten Überblick über die Entwicklertools zu verschaffen.

- Überprüfen Sie im *Inspektor* den Aufbau des Dokuments. Fahren Sie mit dem Mauszeiger über die einzelnen Elemente (Überschrift und Absatz). Klappen Sie testweise den Kopfbereich <head> auf.
- Klicken Sie mit der rechten Maustaste auf den Absatz <p> (Paragraph) und wählen Sie aus dem Kontextmenü *HTML bearbeiten*. Passen Sie den Text an.
- Klicken Sie mit der rechten Maustaste auf den Absatz <p> (Paragraph) und wählen Sie aus dem Kontextmenü *Knoten klonen*. Prüfen Sie das Ergebnis. Wie können Sie den neuen Knoten wieder löschen? Was geschieht bei *Neuen Knoten einfügen*?
- Testen Sie ein paar weitere Möglichkeiten, die das Kontextmenü auf den Elementen des Dokuments zur Verfügung stellt. Es gibt auch verschiedene Varianten zum Kopieren und Einfügen.

- Klicken Sie mit der rechten Maustaste auf den Absatz <p> (Paragraph) und anschliessend im mittleren Unterfenster auf das Pluszeichen. Es wird eine Stylesheet-Regel eingefügt. Ergänzen Sie diese folgendermassen und prüfen Sie das Ergebnis im Browserfenster:

```
p {
  color: goldenrod;
}
```

Alle Änderungen haben wir natürlich nur lokal in der Repräsentation der Seite im Speicher des Browsers durchgeführt. Ein erneutes Laden der Seite setzt sie wieder in den Ausgangszustand zurück. Noch ein paar Bemerkungen zu den Entwicklertools:

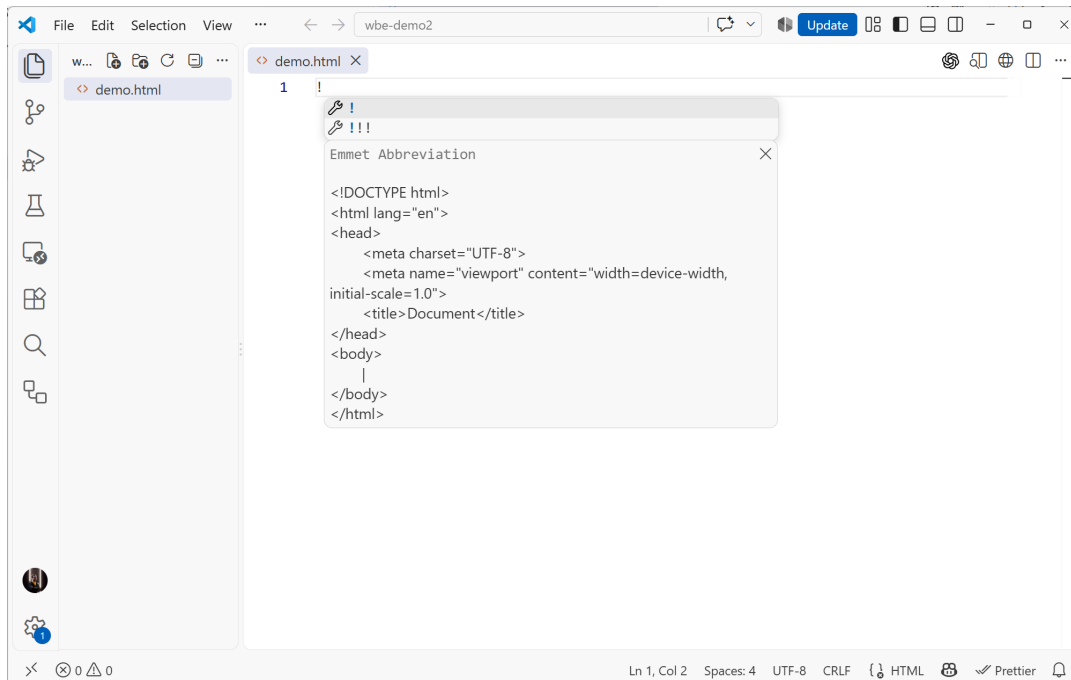
- Neben dem *Inspektor* befindet sich der Eintrag *Konsole*. Die Konsole erlaubt es, JavaScript-Ausdrücke und -Anweisungen einzugeben und auf diese Weise mit der Seite zu interagieren. Wir werden dies in späteren Lektionen noch benötigen. Für den Moment können Sie einmal folgendes ausprobieren:

```
> 2 ** 256
> 2n ** 256n
```

- Die Entwicklertools können an verschiedenen Stellen im Browserfenster platziert oder in einem eigenen Fenster angezeigt werden: Menü mit den drei Punkten auf der rechten Seite.
- Dort finden Sie auch eine Möglichkeit zum Öffnen der Dokumentation zu den Entwicklertools
- Ebenfalls in diesem Menü können die Einstellungen geöffnet werden. Schauen Sie die möglichen Einstellungen durch. Aktivieren Sie mindestens noch das Werkzeug DOM (falls es nicht bereits aktiviert ist), denn auch dieses werden wir in einer späteren Lektion benötigen.

Aufgabe 2: Web-Editor

Legen Sie eine neue leere HTML-Datei im Code-Editor an, zum Beispiel *demo.html*. Die folgenden Angaben beziehen sich auf den Editor Visual Studio Code.



- Geben Sie ein Ausrufezeichen auf der leeren Seite ein. Emmet schlägt daraufhin vor, das Gerüst einer HTML-Seite einzufügen (s. Bild). Emmet kann das Eingeben von HTML- (und teilweise CSS-) Code deutlich beschleunigen, wenn man die passenden Abkürzungen kennt. In VSCode ist es vorinstalliert, in den meisten anderen Code-Editoren lässt es sich nachrüsten (<https://emmet.io>).
- Ergänzen Sie noch den Titel des Dokuments und fügen Sie im Inhaltsbereich (`<body>`) noch eine Überschrift und einen Absatz hinzu. Wenn Sie gar kein HTML kennen, können Sie sich am Beispiel in der letzten Aufgabe orientieren. Speichern Sie das Dokument.
- Betrachten Sie noch die Leiste ganz unten im Editor. Dort ist eingestellt, wie beim Drücken der Tab-Taste eingerückt werden soll (im Bild: *Leerzeichen: 4*), welche Zeichencodierung eingestellt ist (*UTF-8*), welches Zeilenendezeichen eingestellt ist (*LF* für Linefeed) und welche Syntax für das aktuelle Dokument verwendet wird (hier: *HTML*).
- In der Symbolleiste auf der linken Seite finden Sie ganz oben den *Explorer* mit den Dateien des Projekts und weiter unten *Erweiterungen*, wo Sie sich über installierte Erweiterungen informieren und zusätzliche Erweiterungen suchen und installieren können.

Eine grundlegende Einführung in den VSCode ist an dieser Stelle nicht möglich. Es gilt: «Learning by doing». Für Interessierte gibt es hier einige einführende Artikel:

<https://www.microsoft.com/de-de/techwiese/know-how/visual-studio-code-01-die-grundlagen.aspx>

Aufgabe 3: SSH, SFTP

Wichtig – Netzzugang: Der Server *dublin.zhaw.ch* ist nur aus dem ZHAW-Netz oder via VPN erreichbar. Stellen Sie vor dieser und der nächsten Aufgabe sicher, dass Sie im ZHAW-Netz arbeiten oder eine VPN-Verbindung aufgebaut haben. Falls das gerade nicht möglich ist, können Sie die Aufgaben 3 und 4 überspringen und später nachholen – für die weiteren Praktika genügt der lokale Rechner.

Beim ersten Einloggen auf dem *dublin.zhaw.ch* werden einige Einstellungen und Berechtigungen gesetzt. Daher wird in den folgenden Schritten zunächst SSH und dann SFTP verwendet.

- Loggen Sie sich via SSH auf dem *dublin.zhaw.ch* ein. Selbstverständlich müssen Sie statt „bamc“ in den folgenden Schritten Ihr Kurzzeichen eingeben. Es öffnet sich eine Kommandozeile, in der Sie Unix-Befehle eingeben können.

```
~ $ ssh bamc@dublin.zhaw.ch
bamc@dublin.zhaw.ch's password:
Last login: Thu Sep 3 16:49:13 2020
[bamc@dublin ~]$
[bamc@dublin ~]$ ls
private public www
[bamc@dublin ~]$
[bamc@dublin ~]$ exit
```

Bei der ersten Verbindungsaufnahme mit dem Server müssen Sie den Fingerabdruck des Servers bestätigen, um sicherzustellen, dass Sie mit dem korrekten Server verbunden sind. Dieser Fingerabdruck kann in verschiedenen Codierungen angezeigt werden.

- Kopieren Sie die in der letzten Aufgabe erstellte HTML-Datei ins Verzeichnis `www` in Ihrem Home-Verzeichnis auf dem dublin-Server. Verwenden Sie `sftp` auf der Kommandozeile oder einen grafischen SFTP-Client (wie FileZilla, Hinweis dazu s.o.).
- Öffnen Sie Ihre HTML-Datei via HTTP im Browser.

Aufgabe 4: SSH

Auf dem Server Dateien und Verzeichnisse anlegen oder löschen, Dateien anlegen, bearbeiten, verschieben, umbenennen, oder deren Berechtigungen ändern, das sind Aufgaben, die Sie mit einer Remote Shell erledigen können.

- Öffnen Sie erneut eine SSH-Verbindung zum `dublin.zhaw.ch`.
- Wechseln Sie ins Verzeichnis `www` und prüfen Sie, ob die Datei, welche Sie in der letzten Aufgabe angelegt haben, auffindbar ist. Die wichtigsten Kommandos auf der Unix-Shell sollte jeder Informatiker kennen. Bei Bedarf finden Sie im Internet zahlreiche Tutorials. Ein Cheat Sheet mit einigen Kommandos ist als Datei `unix101.pdf` bei den Praktikumsunterlagen zu finden.
- Öffnen Sie Ihre HTML-Datei mit dem `vi`, einem Editor, der auf der Kommandozeile verwendet werden kann (tatsächlich wird die verbesserte Variante `vim` geöffnet):
`vi demo.html`
 Machen Sie ein paar Änderungen in der HTML-Datei und laden sie diese neu im Browser.

Der `vi` gilt als schwierig zu verwenden, eine Handvoll Kommandos genügen aber für die meisten Fälle: Mit der Taste `i` wechseln Sie in den Einfügemodus und können nun Text eingeben, den Cursor platzieren Sie dazu mit den Pfeiltasten, mit **ESC** wechseln Sie zurück in den Kommandomodus, mit `:w` speichern Sie die Datei und mit `:q` beenden Sie den Editor. Auch hier gilt: man sollte mindestens die benötigten Kommandos kennen, um schnell ein paar Änderungen an einer Datei auf dem Server machen zu können. Kleine Hilfe: `vi_cheat_sheet.pdf` bei den Unterlagen.

```

<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Meine erste Seite</title>
</head>
<body>
  <h1>Hallo WBE!</h1>
  <p>Diese Seite wurde im Praktikum 1 erstellt.</p>
</body>
</html>
~
~
~
~
~
~
~
~
~
~
"demo.html" 12L, 288B 10,5 ALL

```

Aufgabe 5: HTTP

Abschliessend noch zwei weitere Kommandos, *curl* und *telnet*. Wenn diese im Terminal Ihres Systems nicht zur Verfügung stehen, können Sie die folgenden Schritte einfach auf dem *dublin* ausführen, zu dem Sie ja problemlos eine SSH-Verbindung herstellen können.

- Stellen Sie mit *host* die IP-Adresse des Servers *dublin.zhaw.ch* fest:

```
host dublin.zhaw.ch
```

- Verwenden Sie *curl*, um Ihre HTML-Datei *demo.html* via HTTP vom Server zu laden:

```
curl https://dublin.zhaw.ch/...
```

Mit *curl* können Sie nicht nur Dateien via HTTP laden: neben GET lassen sich auch andere HTTP-Kommandos verwenden, etwa auch Formulardaten zum Server schicken. Zudem spricht curl auch zahlreiche andere Protokolle (<https://ec.haxx.se>).

Nun noch *telnet*: es erlaubt, eine TCP-Verbindung zu einem Server auf einem bestimmten Port herzustellen und textbasiert mit dem Server zu kommunizieren. So können wir direkt mit dem Webserver per HTTP sprechen.

- Stellen Sie eine Verbindung zu Port 80 des *dublin.zhaw.ch* her:

```
telnet dublin.zhaw.ch 80
```

- Senden Sie ein HTTP-Kommando an den Server (die zweite Zeile gibt an, welchen Host Sie ansprechen möchten – so steht es auch in jeder Anfrage, die ein Browser verschickt):

```
GET /~bamc/hallo.html HTTP/1.0
```

```
Host: dublin.zhaw.ch
```

Zum Abschluss müssen Sie zweimal ENTER drücken. Der Server antwortet mit dem HTTP-Header, dann folgt eine Leerzeile und dann ein kurzer HTML-Inhalt. Anschliessend wird die Verbindung unterbrochen.

```
WBFLAT2 x + v
bmc@srv-app-t-001:~$ telnet dublin.zhaw.ch 80
Trying 160.85.67.138...
Connected to srv-app-t-001.zhaw.ch.
Escape character is '^]'.
GET /~bmc/hallo.html HTTP/1.0
Host: dublin.zhaw.ch

HTTP/1.1 301 Moved Permanently
Date: Thu, 10 Sep 2026 17:18:42 GMT
Server: Apache
Location: https://dublin.zhaw.ch/~bmc/hallo.html
Content-Length: 287
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="https://dublin.zhaw.ch/~bmc/hallo.html">here</a>.</p>
</body></html>
Connection closed by foreign host.
bmc@srv-app-t-001:~$
```

Schauen Sie sich die Antwort genau an: Der Server liefert den Status *301 Moved Permanently* und weist im *Location*-Header auf die HTTPS-Adresse der Seite – unverschlüsseltes HTTP wird heute praktisch überall auf HTTPS umgeleitet. Da *telnet* keine Verschlüsselung beherrscht, können wir der Umleitung nicht folgen; die verschlüsselte Variante haben Sie aber bereits oben mit *curl* abgerufen.

Soweit die Einführung in einige wichtige Werkzeuge und Programme, die in der Web-Entwicklung wichtig sind. Im Laufe des Semesters wird es immer wieder Ergänzungen und weitere Hinweise zu dieser Sammlung geben.